

Tillsyn — Server Architecture Addendum

A focused addendum to `tillsyn-project.md`, capturing the architectural shift from Tailscale-default sync to Hylla-hosted E2EE server sync. This document supplements rather than replaces the existing Tillsyn doc — refer to that for the broader product context, pricing model, UI surfaces, repo structure, MCP integration, and other foundational decisions that remain valid.

Why This Document Exists

The original Tillsyn architecture used Tailscale for both personal multi-device sync and team sync. After working through real-world adoption scenarios — particularly OSS contributors who can't be expected to set up Tailscale to participate in a project — we identified that Tailscale-as-required creates a fatal adoption barrier for the use case Tillsyn most wants to serve.

This addendum captures the resulting architectural shift: Hylla, Inc. operates a hosted server that handles personal sync, team sync, and remote command execution, with end-to-end encryption guaranteeing that Hylla, Inc. cannot read user data even though it operates the infrastructure. Tailscale becomes an advanced optional configuration for users who want it, not a requirement.

The Refined Hard Rule

The original framing was "Hylla, Inc. operates no infrastructure for user/team data." That was wrong — it conflated two different concerns. The actual hard rule, restated:

Hylla, Inc. cannot read user data. This is enforced cryptographically, not by policy.

This permits Hylla to operate infrastructure that handles ciphertext, as long as the encryption boundary is preserved by the protocol. Same trust model Signal operates under: real servers, run by Signal, necessary for the product, but cryptographically incapable of reading messages. Users don't have to trust Signal's good intentions; the math prevents the abuse.

We do the same. The server code is open source. Anyone can audit it and confirm that data flowing through is opaque to us. The protocol is verifiable. This is a stronger trust posture than "trust us, we have a privacy policy" — it's "you don't need to trust us; the math

prevents it."

Unified Architecture

One server, one protocol, one infrastructure, serving personal sync, team sync, and remote command execution.

Hylla-operated team-server (run by Hylla, Inc., open source)

- └ Stores E2EE ciphertext blobs indexed by scope ID + Lamport timestamp
- └ Routes blobs over persistent WebSocket connections to active clients
- └ Wakes sleeping mobile clients via APNs/FCM when needed
- └ Handles auth (knows which user is which, doesn't see data)
- └ Handles billing for team subscriptions
- └ Verifiable via public source that data is opaque to it

Each Tillsyn install (laptop, phone, desktop daemon, etc.):

- └ Opens persistent WebSocket to server when foregrounded
- └ Holds encryption keys locally in OS secure storage
- └ Encrypts before sending; decrypts after receiving
- └ Maintains local SQLite for offline use and CRDT state
- └ Subscribes to relevant scopes

Scopes

A scope is the unit of E2EE-isolated data. Each scope is a separate CRDT document with its own encryption key. The server routes ciphertext blobs by scope ID; clients hold keys for the scopes they're authorized to participate in.

Personal scope: key derived from the user's master key, present only on their own devices. Their phone, laptop, home server all encrypt to the same personal key. Tasks, lists, notes that are theirs alone live here.

Team scope: team key shared among current team members. Each team has its own scope and its own key. New members receive the key during the invite flow; departed members retain whatever they had before they left but stop receiving new updates.

Command scope: specific to a single user; carries action requests and results between their own devices. Used for the remote command execution feature (see below). Typically tied to a designated "execution endpoint" device — a desktop or home server that's set up to receive commands.

A user can have one personal scope, any number of team scopes, and any number of command scopes (typically one per execution endpoint device they own), all syncing

through the same WebSocket connection to the server.

Sync Mechanics

Modern persistent connection model — the same pattern Discord, Slack, Linear, and other real-time apps use. The server doesn't poll clients; clients don't poll the server. A connection is opened once and stays open.

App lifecycle:

1. Tillsyn starts (foreground): opens authenticated WebSocket to server, sends list of subscribed scopes and current state vectors
2. Server responds with everything since the client last synced — encrypted blobs, delivered immediately
3. Connection stays open; heartbeats every ~30-60 seconds prevent firewall/NAT timeouts
4. When any other device on any subscribed scope sends an update, server pushes the ciphertext over this open connection
5. Client decrypts, applies CRDT update locally, displays in UI
6. Client's own edits get encrypted and sent up the same connection
7. Server fans out to all other subscribed devices currently connected

When mobile backgrounds:

1. OS suspends the app, kills the WebSocket within ~1-2 minutes
2. Server marks the device offline; queues messages for it
3. When something arrives that should reach the device, server sends a wake signal via APNs (iOS) or FCM (Android)
4. APNs/FCM delivers a small notification to the OS; if Tillsyn is configured for it, brief background time runs
5. Tillsyn reconnects WebSocket, fetches queued messages
6. When user opens the app again, full sync resumes

APNs/FCM see metadata only — that a notification was delivered to a device at a specific time, plus whatever notification text Tillsyn chooses to include. They never see the actual encrypted payload.

CRDT layer for offline edits: if the device made edits while disconnected, those merge

cleanly with whatever happened in the meantime. CRDTs are commutative and associative; merge order doesn't matter. This is what makes "offline edits sync correctly when reconnected" feel automatic.

Onboarding (No Tailscale, No Infrastructure Setup)

Solo user, multiple devices:

1. Install Tillsyn on laptop, sign in (or create account)
2. Install Tillsyn on phone, sign in to same account
3. Devices sync automatically via personal scope
4. Done — total time under a minute

OSS maintainer creating a team:

1. Already using Tillsyn from above
2. Click "Create team," name it, pay the team fee through Stripe Checkout
3. Team scope is created; encryption key generated client-side; key registered with the server (encrypted to the maintainer's account so server still can't read it)
4. Generate invite link, share in project README/Discord/Slack/wherever
5. Done

OSS contributor joining:

1. Click invite link on any device
2. Tillsyn opens (or installs, if needed)
3. Sign in with GitHub (or Google, Apple — whatever auth options we support)
4. Invite link contains the team key encrypted with a one-time secret; Tillsyn decrypts and stores the team key locally
5. Tillsyn registers with the server as a member of that team scope
6. Sync starts; team's CRDT data flows down; member sees the project's tasks
7. Done — total time under 2 minutes for a contributor

No Tailscale anywhere in the default flow. No port forwarding. No "set up your own server."
No infrastructure setup at all. Just install and sign in.

Encryption Protocol

For v1, simple team-wide symmetric key. Each team has one shared symmetric encryption key (AES-256 or ChaCha20-Poly1305). Members encrypt all data with this key before sending to the server; the server stores ciphertext; recipients decrypt with the same key.

Joining a team: existing member generates an invite. The invite contains the team's symmetric key, encrypted with a one-time secret embedded in the invite link. New member opens the invite, gets the key, can now decrypt team data.

Pros: Simple. Fast. Easy to implement correctly.

Cons: No forward secrecy (if key leaks, all past and future team data is readable until rotated). No clean revocation (removing a member requires rotating the key and re-encrypting). Trust model is "every member sees everything in the team scope."

For most teams, especially OSS where members rarely get "removed" so much as drift away, this is genuinely fine. We're not competing with Signal; we're competing with Asana, Linear, Trello — none of which are E2EE at all.

Migration path to MLS (Messaging Layer Security): v2+ should adopt MLS, the IETF standard for group E2EE. MLS provides forward secrecy, post-compromise security, and clean revocation. It's used by AWS Wickr, RingCentral, and others. Open-source libraries exist (OpenMLS). The data model designed for v1 (each scope has a key controlled by members rather than the server) supports a clean migration: swap the key derivation; everything else stays the same.

For v1: ship simple symmetric. For v2: migrate to MLS.

Remote Command Execution

Beyond sync, the persistent connection architecture enables a powerful pattern: trigger actions on a remote device. Used elsewhere as "Claude Code remote," "remote control," or "command-and-control over messaging." For Tillsyn this becomes a native feature.

The use case: user moves a task from "todo" to "in progress" on their phone. The desktop Tillsyn instance receives the command, triggers an associated workflow — opens the project directory in their IDE, launches a tmux session, kicks off Claude Code subagents for the task, posts a Slack status update — all according to user-defined automation. The mobile app is a thin remote control; the desktop does the actual work.

How it works architecturally: rides on the same WebSocket sync infrastructure. Phone sends an encrypted "execute X" message to the user's command scope. Server routes the ciphertext to the user's desktop daemon (which is subscribed to that scope via its open

WebSocket). Desktop decrypts, executes, encrypts results, sends back. Phone receives results.

The server sees only ciphertext throughout. Hylla, Inc. cannot see what was executed or what the result was.

Comparison to Claude Code remote: the architectural pattern is the same. Tillsyn does this slightly more rigorously by making encryption explicit and end-to-end so the relay (the server) is cryptographically incapable of reading commands or results — not just policy-bound.

Security model is delegated to existing tools: Tillsyn doesn't try to sandbox arbitrary command execution itself. The expectation is that users invoke Tillsyn alongside Claude Code or similar tools that handle their own sandboxing for the actual work. Tillsyn provides the trigger and the routing; the executing tool provides the safety guarantees for what runs. This keeps Tillsyn focused on its core competency (task and team coordination) and avoids reinventing security infrastructure that's better handled by purpose-built tools.

v1.1 scope: initial implementation supports user-defined trigger workflows (move task to "in progress," push notification, run pre-defined script). Arbitrary remote code execution from the phone is not the goal; integration with existing tools that handle the heavy lifting is.

Audit trail: every remote command execution is logged locally on the desktop with timestamp, source device, action triggered, and outcome. The user can review what was run remotely. Logs are local; not sent to Hylla.

Sleeping desktop: if the user's desktop is offline when the command is sent, the command queues at the server (encrypted). When the desktop comes online, the queued command delivers. Users can configure max-age policies (e.g., "discard commands older than 4 hours") to prevent stale commands from running unexpectedly.

What Tailscale Becomes

Tailscale stops being required for anything. It becomes an advanced optional configuration for users who specifically want their data to never touch any Hylla-operated infrastructure:

Tailscale-only mode (advanced opt-in): No hosted server in the path. Direct peer-to-peer over Tailscale. Best for security-extreme users or small teams. Original architecture remains available, documented as advanced. The trust posture is even stronger here — no Hylla servers see anything, not even ciphertext metadata — but the setup friction returns.

Self-hosted server mode: Same Tillsyn server binary, run by the team or organization. Tailscale optional but useful for the access control story. Serves the corporate-team use case where the org wants their own infrastructure.

Default mode (most users): Hylla-hosted server, E2EE by default, no Tailscale. This is what 99%+ of users get.

Personal sync follows the same pattern — default to the hosted server, optional Tailscale for users who want it.

Server Implementation Notes

The server itself is implementation-wise straightforward, well-suited to Go:

- WebSocket-heavy server with persistent connections
- Each connection is one goroutine; Go's runtime handles this comfortably to 100K+ concurrent connections per machine
- `nhooyr.io/websocket` or `gorilla/websocket` for the WebSocket layer
- Postgres for durable record (account info, scope membership, billing state)
- Redis or NATS for pub/sub between server instances if horizontally scaled
- APNs/FCM integration for mobile wake signals
- Caddy for TLS termination
- Stateless server design behind a load balancer; messages routed via the pub/sub layer

This is well-trodden territory. The pattern matches Discord's gateway, Slack's RTM (historically), and many others.

Cost Implications

Personal sync going through Hylla's server means Hylla pays bandwidth/storage for free users' personal data. This sounds expensive but is not, in practice:

- Personal task data is tiny: text, metadata, no binary attachments in v1
- A typical free user generates maybe 10-100KB of sync traffic per day
- A modest VPS can handle tens of thousands of free users on personal sync
- Team subscriptions subsidize free-tier infrastructure costs

The economics work. Tillsyn isn't a video service; it's a task tracker with low per-user bandwidth. Storage costs are similarly small (CRDT histories compress well; old snapshots can be archived).

Pricing Model — Unchanged

The pricing model from the original Tillsyn doc remains valid:

- **Free for everyone at the user level.** Local use, mobile sync (now via the hosted server, free), team participation (free for joiners) all remain free.
- **Mobile fee:** paid by the individual who wants mobile sync. Now this is what gets the user a Tillsyn account on Hylla's server with mobile-app entitlements; same model.
- **Team fee:** paid once per team by the team creator. Subscription pricing covers Hylla's team-server costs and contributes to development. Members never pay.
- **Joining a team is always free.**

The shift to a hosted server doesn't change who pays. It changes how the infrastructure is delivered.

What This Doesn't Change

Reading this addendum alongside the original Tillsyn doc, most things remain valid:

- Apache 2.0 open source for everything
- Multi-UI architecture (TUI, Electron, mobile, MCP)
- Persistent daemon model on each desktop (now talking to a remote server in addition to local UIs)
- CRDT data model
- Pricing tiers and philosophy
- Profit-sharing tenets
- Community-driven moat thesis
- Repo structure (with addition of a server module)

The architectural change is significant but localized: it's about how devices talk to each other, not about the rest of the product.

Repo Structure Update

Adding the server to the existing Tillsyn monorepo:

tillsyn/

(Apache 2.0, public, owned by Hylla, Inc.)

└─ daemon/	Go: SQLite owner + sync hub + tsnet (now optional) + HTTP
└─ server/	Go: hosted team-server — WebSocket relay, E2EE blob storage
└─ core/	Shared Go: data model, CRDT schemas, encryption protocol
└─ tui/	Go TUI (HTTP client to daemon)
└─ ui/	Astro + Solid (the UI codebase, used by Electron + mobile)
└─ electron/	Electron wrapper; bundles daemon + UI
└─ mobile/	Capacitor app: UI + Capacitor + WebSocket client
└─ plugin/	Capacitor plugin (Swift + Kotlin) wrapping native crypto
└─ docs/	

The server is part of the same repo because:

- It shares the data model and CRDT schemas with the daemon
- The encryption protocol is shared between client and server
- Releases can be coordinated (server-side changes that require client changes ship together)
- The “server” and “daemon” are 90% the same code anyway — they just have different deployment configurations

Self-hosters can build the server from this repo and run it themselves.

Open Questions Specific to This Addendum

- **[ENCRYPTION-V1]** Confirm choice of symmetric cipher (AES-256-GCM vs ChaCha20-Poly1305 vs XChaCha20-Poly1305). Probably XChaCha20-Poly1305 for the larger nonce space, but worth verifying with crypto review.
- **[INVITE-PROTOCOL]** Specific format for invite links and one-time secret protocols. Standard practice exists (Wire, Signal, etc.); pick a documented pattern.
- **[KEY-ROTATION]** v1 doesn’t require key rotation. Document what happens when a team wants to rotate (manually-triggered re-encryption of recent state, full audit logged).
- **[ACCOUNT-AUTH]** Confirm SSO providers (GitHub primary for OSS adoption, Google secondary, Apple for iOS, email/password as fallback). Account-recovery flow specifics.
- **[MCP-AND-COMMAND-SCOPE]** How MCP integration interacts with the command scope feature. Likely clean — MCP exposes tasks; command scope enables triggering automations on transitions — but worth specifying.
- **[SELF-HOSTED-DOCS]** Documentation quality bar for self-hosters. Setup recipes, security guidance, backup procedures, audit configuration. Not v1 priority but should

not be afterthought.

One-Line Summary

Tillsyn moves to a Hylla-operated E2EE server as the default sync infrastructure for personal, team, and remote-command-execution use cases. Trust is preserved cryptographically rather than architecturally — Hylla cannot read user data even though Hylla operates the servers. Tailscale becomes an advanced opt-in for users who specifically want zero Hylla involvement in the data path. Onboarding drops from “set up Tailscale and credentials” to “install and sign in” — under two minutes for OSS contributors joining a project. Same persistent-WebSocket pattern Discord and Slack use; same MLS migration path Signal-class apps follow; same trust model Signal operates under.